

Contact Manager

Complete React + Express CRUD Flow & Learning Guide

Purpose: Repository-ready documentation for understanding how the React frontend, Express backend, API requests, React state, routing, and `contacts.json` data store work together.

Learning level: Fresher / beginner-friendly, while retaining the correct technical terminology.

1. Big Picture

The application has two parts: a React frontend and an Express backend. React controls the UI and state. Express exposes REST API endpoints. The backend uses `contacts.json` as a simple file-based data store instead of a real database.



2. Project Structure

```
contact-manager/
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── ContactCard.jsx
│   │   │   └── pages/
│   │       ├── ContactList.jsx
│   │       ├── ContactDetail.jsx
│   │       └── ContactForm.jsx
│   └── backend/
│       ├── server.js
│       ├── contacts.json
│       └── package.json
```

3. The Most Important Mental Model

`App.jsx` is the central controller.

```
ContactForm ---> addContact/updateContact ---> API ---> server.js ---> contacts.json
ContactCard ---> deleteContact/toggleFavorite ---> API ---> server.js
Navbar ---> React Router navigation
App.jsx ---> passes state + functions to child components
```

```
Backend response ---> response.json() ---> setContacts() ---> React re-render ---> UI
```

4. HTTP Methods Used

Operation	Method	Endpoint	Purpose
Read all	GET	/api/contacts	Fetch all contacts
Read one	GET	/api/contacts/:id	Fetch one contact
Create	POST	/api/contacts	Create a contact
Update	PUT	/api/contacts/:id	Replace/update a contact
Delete	DELETE	/api/contacts/:id	Delete a contact
Favorite	PATCH	/api/contacts/:id/favorite	Toggle favorite

5. React App.jsx — Complete Code

This version uses the backend as the source of persisted data. The contacts state is initially empty, then populated by GET /api/contacts.

```
import { useEffect, useState } from "react";
import { Routes, Route } from "react-router-dom";

import Navbar from "../components/Navbar";
import ContactList from "../pages/ContactList";
import ContactDetail from "../pages/Contactdetail";
import ContactForm from "../pages/ContactForm";

const API_URL = "http://127.0.0.1:5000/api/contacts";

function App() {
  const [contacts, setContacts] = useState([]);

  // Get contacts from backend
  useEffect(() => {
    const fetchContacts = async () => {
      try {
        const response = await fetch(API_URL);

        if (!response.ok) {
          throw new Error("Failed to fetch contacts");
        }

        const data = await response.json();
        setContacts(data);
      } catch (error) {
        console.error("Error fetching contacts:", error);
      }
    };

    fetchContacts();
  }, []);

  // M3 - Add Contact
  const addContact = async (newContact) => {
    try {
      const response = await fetch(API_URL, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(newContact),
      });

      if (!response.ok) {
        throw new Error("Failed to add contact");
      }

      const contactWithId = await response.json();

      setContacts((previousContacts) => [
        ...previousContacts,
        contactWithId,
      ]);
    } catch (error) {
      console.error("Error adding contact:", error);
    }
  };

  // M4 - Update Contact
  const updateContact = async (updatedContact) => {
    try {
      const response = await fetch(
        `${API_URL}/${updatedContact.id}`,
        {
          method: "PUT",
          headers: {
            "Content-Type": "application/json",
          },
          body: JSON.stringify(updatedContact),
        }
      );
    }
  };
}
```

```

    if (!response.ok) {
      throw new Error("Failed to update contact");
    }

    const savedContact = await response.json();

    setContacts((previousContacts) =>
      previousContacts.map((contact) =>
        contact.id === savedContact.id
          ? savedContact
          : contact
      )
    );
  } catch (error) {
    console.error("Error updating contact:", error);
  }
};

// M4 - Delete Contact
const deleteContact = async (id) => {
  try {
    const response = await fetch(
      `${API_URL}/${id}`,
      {
        method: "DELETE",
      }
    );

    if (!response.ok) {
      throw new Error("Failed to delete contact");
    }

    setContacts((previousContacts) =>
      previousContacts.filter(
        (contact) => contact.id !== id
      )
    );
  } catch (error) {
    console.error("Error deleting contact:", error);
  }
};

// M5 - Toggle Favorite
const toggleFavorite = async (id) => {
  try {
    const response = await fetch(
      `${API_URL}/${id}/favorite`,
      {
        method: "PATCH",
      }
    );

    if (!response.ok) {
      throw new Error("Failed to update favorite");
    }

    const updatedContact = await response.json();

    setContacts((previousContacts) =>
      previousContacts.map((contact) =>
        contact.id === updatedContact.id
          ? updatedContact
          : contact
      )
    );
  } catch (error) {
    console.error("Error updating favorite:", error);
  }
};

return (
  <>
    <Navbar />

    <Routes>
      <Route
        path="/"
        element={
          <ContactList
            contacts={contacts}

```

```

        onDelete={deleteContact}
        onToggleFavorite={toggleFavorite}
      />
    }
  />

  <Route
    path="/add"
    element={
      <ContactForm
        onAdd={addContact}
      />
    }
  />

  <Route
    path="/edit/:id"
    element={
      <ContactForm
        contacts={contacts}
        onUpdate={updateContact}
      />
    }
  />

  <Route
    path="/contact/:id"
    element={
      <ContactDetail
        contacts={contacts}
      />
    }
  />
</Routes>
</>
);
}

export default App;

```

How App.jsx connects to the UI

contacts is state. App passes it to ContactList and ContactDetail. App also passes functions as props: onAdd, onUpdate, onDelete, and onToggleFavorite. Child components call these functions when the user performs an action.

Important React state pattern

```

setContacts((previousContacts) => [
  ...previousContacts,
  contactWithId,
]);

```

The spread operator copies the existing array elements into a new array. It is not the rest operator here. The result is a new array containing the old contacts plus the new contact.

6. ContactForm.jsx — Complete Add/Edit Form

```
import { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";

const ContactForm = ({
  onAdd,
  contacts = [],
  onUpdate,
}) => {
  const navigate = useNavigate();
  const { id } = useParams();
  const isEditMode = Boolean(id);

  // Find the contact to edit
  const contactToEdit = contacts.find(
    (contact) => contact.id === Number(id)
  );

  const [formData, setFormData] = useState({
    name: contactToEdit?.name || "",
    email: contactToEdit?.email || "",
    phone: contactToEdit?.phone || "",
    city: contactToEdit?.city || "",
  });

  const [errors, setErrors] = useState({});

  const nameRegex = /^[A-Za-z ]+$/;
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const phoneRegex = /^[0-9]{10}$/;
  const cityRegex = /^[A-Za-z ]+$/;

  // Important when backend data loads asynchronously
  useEffect(() => {
    if (isEditMode) {
      const contact = contacts.find(
        (contact) => contact.id === Number(id)
      );

      if (contact) {
        setFormData({
          name: contact.name,
          email: contact.email,
          phone: contact.phone,
          city: contact.city,
        });
      }
    }
  }, [id, contacts, isEditMode]);

  const handleChange = (e) => {
    const { name, value } = e.target;

    setFormData((previousData) => ({
      ...previousData,
      [name]: value,
    }));
  };

  const validateForm = () => {
    const newErrors = {};

    if (!formData.name.trim()) {
      newErrors.name = "Name is required";
    } else if (!nameRegex.test(formData.name)) {
      newErrors.name = "Name should contain only letters";
    }

    if (!formData.email.trim()) {
      newErrors.email = "Email is required";
    } else if (!emailRegex.test(formData.email)) {
      newErrors.email = "Enter a valid email";
    }

    if (!formData.phone.trim()) {
      newErrors.phone = "Phone number is required";
    } else if (!phoneRegex.test(formData.phone)) {
      newErrors.phone = "Phone number must contain exactly 10 digits";
    }
  };
};
```

```

    }

    if (!formData.city.trim()) {
      newErrors.city = "City is required";
    } else if (!cityRegex.test(formData.city)) {
      newErrors.city = "City should contain only letters";
    }

    setErrors(newErrors);

    return Object.keys(newErrors).length === 0;
  };

const handleSubmit = (e) => {
  e.preventDefault();

  const isValid = validateForm();

  if (!isValid) {
    return;
  }

  if (isEditMode) {
    onUpdate({
      ...formData,
      id: Number(id),
    });
  } else {
    onAdd(formData);
  }

  navigate("/");
};

return (
  <div className="max-w-xl mx-auto mt-10 p-6 bg-white shadow-md rounded-lg">
    <h1 className="text-2xl font-bold mb-6">
      {isEditMode ? "Edit Contact" : "Add Contact"}
    </h1>

    <form onSubmit={handleSubmit} className="space-y-4">
      <div>
        <label className="block mb-1 font-medium">Name</label>
        <input
          type="text"
          name="name"
          placeholder="Enter name"
          value={formData.name}
          onChange={handleChange}
          className="w-full border p-2 rounded"
        />
        {errors.name && (
          <p className="text-red-500 text-sm mt-1">
            {errors.name}
          </p>
        )}
      </div>

      <div>
        <label className="block mb-1 font-medium">Email</label>
        <input
          type="text"
          name="email"
          placeholder="Enter email"
          value={formData.email}
          onChange={handleChange}
          className="w-full border p-2 rounded"
        />
        {errors.email && (
          <p className="text-red-500 text-sm mt-1">
            {errors.email}
          </p>
        )}
      </div>

      <div>
        <label className="block mb-1 font-medium">Phone</label>
        <input
          type="text"
          name="phone"

```

```

        placeholder="Enter 10-digit phone number"
        value={formData.phone}
        onChange={handleChange}
        className="w-full border p-2 rounded"
      />
      {errors.phone && (
        <p className="text-red-500 text-sm mt-1">
          {errors.phone}
        </p>
      )}
    </div>

    <div>
      <label className="block mb-1 font-medium">City</label>
      <input
        type="text"
        name="city"
        placeholder="Enter city"
        value={formData.city}
        onChange={handleChange}
        className="w-full border p-2 rounded"
      />
      {errors.city && (
        <p className="text-red-500 text-sm mt-1">
          {errors.city}
        </p>
      )}
    </div>

    <button
      type="submit"
      className="w-full bg-blue-600 text-white py-2 rounded"
    >
      {isEditMode ? "Update Contact" : "Add Contact"}
    </button>
  </form>
</div>
);
};

export default ContactForm;

```

M3 vs M4 in ContactForm

M3 Add: The form is normally blank. After validation, `onAdd(formData)` calls `App.addContact()`, which sends POST to the backend.

M4 Edit: The URL contains `/edit/:id`. `useParams()` gets the ID, the existing contact is found, `formData` is populated, and `onUpdate({ ...formData, id })` sends PUT to the backend.

Why `contactToEdit?.name || ""`?

Optional chaining (`?.`) prevents an error when `contactToEdit` does not exist. The `|| ""` fallback makes the input blank when there is no existing value.

7. ContactCard.jsx — View, Edit, Delete, Favorite

```
import { Link } from "react-router-dom";

const ContactCard = ({
  contact,
  onDelete,
  onToggleFavorite,
}) => {
  const handleDelete = () => {
    const confirmed = window.confirm(
      "Are you sure you want to delete this contact?"
    );

    if (confirmed) {
      onDelete(contact.id);
    }
  };

  return (
    <div className="bg-white shadow-md rounded-lg p-5">
      <div className="flex justify-between items-start">
        <h2 className="text-xl font-bold mb-3">
          {contact.name}
        </h2>

        <button
          onClick={() => onToggleFavorite(contact.id)}
          className="text-2xl"
          title="Toggle favorite"
        >
          {contact.favorite ? "★" : "■"}
        </button>
      </div>

      <p>Email: {contact.email}</p>
      <p>Phone: {contact.phone}</p>
      <p>City: {contact.city}</p>

      <div className="flex gap-3 mt-5">
        <Link
          to={`/contact/${contact.id}`}
          className="bg-green-600 text-white px-3 py-2 rounded"
        >
          View
        </Link>

        <Link
          to={`/edit/${contact.id}`}
          className="bg-blue-600 text-white px-3 py-2 rounded"
        >
          Edit
        </Link>

        <button
          onClick={handleDelete}
          className="bg-red-600 text-white px-3 py-2 rounded"
        >
          Delete
        </button>
      </div>
    </div>
  );
};

export default ContactCard;
```

Why View/Edit use Link but Delete uses onClick

Link is for navigation: View goes to `/contact/:id` and Edit goes to `/edit/:id`. Delete is not navigation; it must execute confirmation and then call the delete function, so it uses a button with `onClick`.

8. Navbar.jsx

```
import { Link } from "react-router-dom";
```

```

const Navbar = () => {
  return (
    <nav className="w-full bg-blue-600 shadow-md">
      <div className="max-w-7xl mx-auto px-6 py-4 flex items-center justify-between">
        <Link
          to="/"
          className="text-2xl font-bold text-white"
        >
          Contact Manager
        </Link>

        <div className="flex items-center gap-6">
          <Link
            to="/"
            className="text-white font-medium"
          >
            Home
          </Link>

          <Link
            to="/add"
            className="bg-white text-blue-600 px-4 py-2 rounded-lg font-medium"
          >
            Add Contact
          </Link>
        </div>
      </div>
    </nav>
  );
};

export default Navbar;

```

Navigation

Link to="/" navigates to the Home route without a traditional browser page reload. **navigate("/")** is useful after an action such as a successful Add or Update.

9. server.js — Complete Backend

```
const express = require("express");
const cors = require("cors");
const fs = require("fs");
const path = require("path");

const app = express();

app.use(cors());
app.use(express.json());

const PORT = 5000;
const contactsFile = path.join(__dirname, "contacts.json");

// Read contacts from contacts.json
const getContacts = () => {
  const data = fs.readFileSync(contactsFile, "utf-8");
  return JSON.parse(data);
};

// Save contacts to contacts.json
const saveContacts = (contacts) => {
  fs.writeFileSync(
    contactsFile,
    JSON.stringify(contacts, null, 2)
  );
};

// GET all contacts
app.get("/api/contacts", (req, res) => {
  const contacts = getContacts();
  res.json(contacts);
});

// GET one contact
app.get("/api/contacts/:id", (req, res) => {
  const { id } = req.params;
  const contacts = getContacts();

  const contact = contacts.find(
    (contact) => contact.id === Number(id)
  );

  if (!contact) {
    return res.status(404).json({
      message: "Contact not found",
    });
  }

  res.json(contact);
});

// CREATE contact
app.post("/api/contacts", (req, res) => {
  const contacts = getContacts();

  const newContact = {
    id: contacts.length > 0
      ? Math.max(...contacts.map((contact) => contact.id)) + 1
      : 1,
    name: req.body.name,
    email: req.body.email,
    phone: req.body.phone,
    city: req.body.city,
    favorite: false,
  };

  contacts.push(newContact);
  saveContacts(contacts);

  res.status(201).json(newContact);
});

// UPDATE contact
app.put("/api/contacts/:id", (req, res) => {
  const { id } = req.params;
  const contacts = getContacts();
```

```

const contactIndex = contacts.findIndex(
  (contact) => contact.id === Number(id)
);

if (contactIndex === -1) {
  return res.status(404).json({
    message: "Contact not found",
  });
}

contacts[contactIndex] = {
  ...contacts[contactIndex],
  ...req.body,
  id: Number(id),
};

saveContacts(contacts);
res.json(contacts[contactIndex]);
});

// DELETE contact
app.delete("/api/contacts/:id", (req, res) => {
  const { id } = req.params;
  const contacts = getContacts();

  const updatedContacts = contacts.filter(
    (contact) => contact.id !== Number(id)
  );

  if (updatedContacts.length === contacts.length) {
    return res.status(404).json({
      message: "Contact not found",
    });
  }

  saveContacts(updatedContacts);

  res.json({
    message: "Contact deleted successfully",
  });
});

// TOGGLE favorite
app.patch("/api/contacts/:id/favorite", (req, res) => {
  const { id } = req.params;
  const contacts = getContacts();

  const contact = contacts.find(
    (contact) => contact.id === Number(id)
  );

  if (!contact) {
    return res.status(404).json({
      message: "Contact not found",
    });
  }

  contact.favorite = !contact.favorite;

  saveContacts(contacts);
  res.json(contact);
});

// Start server
app.listen(PORT, "127.0.0.1", () => {
  console.log(
    `Server running on http://127.0.0.1:${PORT}`
  );
});

```

Backend imports

express creates the API server and routes. **cors** permits frontend/backend cross-origin requests. **fs** reads and writes `contacts.json`. **path** builds the path to `contacts.json`.

Middleware

app.use(cors()) enables CORS middleware. **app.use(express.json())** parses JSON request bodies so POST/PUT data is available through `req.body`.

getContacts()

```
const getContacts = () => {
  const data = fs.readFileSync(contactsFile, "utf-8");
  return JSON.parse(data);
};
```

`fs.readFileSync` reads the file as text. `JSON.parse` converts that JSON text into a JavaScript array/object.

saveContacts()

```
const saveContacts = (contacts) => {
  fs.writeFileSync(
    contactsFile,
    JSON.stringify(contacts, null, 2)
  );
};
```

`JSON.stringify` converts JavaScript data into JSON text. The `null` means no property filtering/replacement. The `2` means two-space indentation for readable JSON. `writeFileSync` persists the text to `contacts.json`.

10. M3 Add Contact — Complete Flow

```
User fills ContactForm
  |
  | onSubmit
  v
validateForm()
  |
  | valid
  v
onAdd(formData)
  |
  v
App.addContact(newContact)
  |
  | POST /api/contacts
  | JSON.stringify(newContact)
  v
Express app.post()
  |
  | req.body
  v
Create newContact + generated ID
  |
  v
contacts.push(newContact)
  |
  v
saveContacts(contacts)
  |
  v
contacts.json
  |
  v
res.status(201).json(newContact)
  |
  v
const contactWithId = await response.json()
  |
  v
setContacts([...previousContacts, contactWithId])
  |
  v
React re-render
  |
  v
New contact appears in UI
```

Important: contactWithId

`contactWithId` is not only the ID. It is the entire contact object returned by the backend, including the ID generated by the backend.

Important: push()

`contacts.push(newContact)` adds the contact to the in-memory JavaScript array. `saveContacts(contacts)` is what persists the updated array to `contacts.json`. This project therefore has a JSON file data store, not a real database.

11. M4 Update — Complete Flow

```
Click Edit
|
v
<Link to={`/edit/${contact.id}`}>
|
v
URL: /edit/3
|
v
React Router matches /edit/:id
|
v
ContactForm
|
| useParams() -> id = "3"
|
v
Find contact with ID 3
|
v
Populate formData
|
v
User changes fields
|
v
handleChange()
|
v
handleSubmit()
|
v
onUpdate({...formData, id: Number(id)})
|
v
App.updateContact()
|
| PUT /api/contacts/3
|
v
Express app.put("/api/contacts/:id")
|
v
Find matching array index
|
v
Merge old contact + req.body
|
v
saveContacts()
|
v
res.json(saved contact)
|
v
React map() replaces only matching contact
|
v
UI re-renders
```

Why map() for update?

map() creates a new array. For every contact, if the ID matches the saved contact ID, it returns the updated contact; otherwise it returns the existing contact.

```
setContacts((previousContacts) =>
  previousContacts.map((contact) =>
    contact.id === savedContact.id
      ? savedContact
      : contact
  )
);
```

12. M4 Delete — Complete Flow

```
Click Delete
|
v
onClick={handleDelete}
|
v
window.confirm()
|
| confirmed
v
onDelete(contact.id)
|
v
App.deleteContact(id)
|
| DELETE /api/contacts/:id
v
Express delete route
|
v
filter() removes matching ID
|
v
saveContacts(updatedContacts)
|
v
contacts.json updated
|
v
React filter() removes same contact from state
|
v
UI re-renders
```

Why filter() for delete?

`filter()` keeps every contact whose ID is different from the ID being deleted. The result is a new array without the deleted contact.

```
setContacts((previousContacts) =>
  previousContacts.filter(
    (contact) => contact.id !== id
  )
);
```

13. IDs — Why Deleting a Contact Does Not Break the Generator

```
id: contacts.length > 0
  ? Math.max(...contacts.map((contact) => contact.id)) + 1
  : 1,
```

The backend does not simply use `contacts.length + 1`. It first extracts all IDs, finds the largest ID, and adds one.

```
Existing IDs: [1, 3, 4]
Math.max(1, 3, 4) = 4
4 + 1 = 5
```

```
Next ID = 5
```

Therefore deleting ID 2 does not cause the next ID to become 4. If the highest ID itself is deleted, the current implementation can reuse that ID later. For a production system, use a database-generated identity/auto-increment ID or UUID.

14. Regex Validation

```
const nameRegex = /^[A-Za-z ]+$/;
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phoneRegex = /^[0-9]{10}$/;
const cityRegex = /^[A-Za-z ]+$/;
```

Name/City: letters and spaces only. **Email:** basic structure containing a local part, @, domain, and dot. **Phone:** exactly 10 digits. The phone regex shown here allows any first digit; if the project specifically requires Indian mobile numbers, use a stricter rule.

15. Template Literals and URL IDs

```
`${API_URL}/${updatedContact.id}`
```

Backticks create a template literal. `${...}` inserts a JavaScript value into the string. The `/` is the URL path separator.

```
API_URL = http://127.0.0.1:5000/api/contacts
updatedContact.id = 3
```

```
Result:
http://127.0.0.1:5000/api/contacts/3
```

16. response.json() and JSON.stringify()

JSON.stringify() — frontend to backend

```
body: JSON.stringify(newContact)
```

Converts a JavaScript object into JSON text so it can be sent in the HTTP request body.

response.json() — backend to frontend

```
const contactWithId = await response.json();
```

Reads the JSON response body and converts it into a JavaScript object. In the Add flow, that object is the complete newly created contact returned by the backend.

17. Full Add Request/Response Example

Frontend sends:

```
POST /api/contacts
Content-Type: application/json
```

```
{
  "name": "Aarav",
  "email": "aarav@mail.com",
  "phone": "9876543210",
  "city": "Pune"
}
```

Backend creates:

```
{
  "id": 5,
  "name": "Aarav",
  "email": "aarav@mail.com",
  "phone": "9876543210",
  "city": "Pune",
  "favorite": false
}
```

Backend response:

```
201 Created
```

React receives:

```
const contactWithId = await response.json();
```

Then:

```
setContacts((previousContacts) => [
  ...previousContacts,
  contactWithId,
]);
```

18. Running the Project

Backend

```
cd backend
npm install
node server.js
```

Expected output: Server running on http://127.0.0.1:5000

Frontend

```
cd frontend
npm install
npm run dev
```

Open the Vite URL shown in the terminal, commonly http://localhost:5173.

Important

Do not use VS Code Code Runner to execute individual React JSX files as the application. Run the React project with its Vite/npm script. Run the Express backend with node server.js.

19. Troubleshooting Checklist

Symptom	Check
Failed fetch / Network error	Confirm backend is running on 127.0.0.1:5000.
CORS error	Confirm app.use(cors()) is present and backend restarted.
req.body undefined	Confirm app.use(express.json()) and Content-Type: application/json.
Edit form initially blank	Because contacts arrive asynchronously; useEffect should populate formData after contacts load.
Browser fully reloads	A React state update is normally only a re-render. Check browser Network tab for a real document request and check Vite.
ReferenceError: d is not defined	Check server.js for accidental characters such as 'd' after app.listen().

20. One-Page Learning Summary

React:

```
State = contacts
Form state = formData
Router = Routes + Route + Link + useNavigate/useParams
API = fetch()
```

Backend:

```
Express = server + routes
CORS = frontend/backend permission
express.json() = parse JSON body
fs = read/write file
path = locate file
```

Persistence:

```
contacts.json is the current data store.
push() changes the array in memory.
saveContacts() persists it.
```

Add:

```
POST -> backend creates ID -> response.json() -> setContacts([...])
```

Update:

```
PUT /:id -> backend updates matching record -> map() updates state
```

Delete:

```
DELETE /:id -> backend removes record -> filter() updates state
```

Favorite:

```
PATCH /:id/favorite -> backend toggles boolean -> map() updates state
```

Navigation:

```
Link = navigation
onClick = execute custom action
navigate("/") = programmatic navigation
```

Core principle: The frontend does not directly edit `contacts.json`. It communicates with the backend through HTTP. The backend is responsible for reading/writing the data store, and React updates its UI state from the API results.